IOT PRACTICALS

0.

🎯 Aim

To boot Raspbian OS on Raspberry Pi and understand its hardware components and basic interfacing with monitor, keyboard, mouse, and Ethernet.

Components Required

- Raspberry Pi (Model 3 / 4)
- MicroSD Card (16GB or above)
- Power Adapter (5V, 3A recommended)
- HDMI Cable
- Monitor
- USB Keyboard & Mouse
- Ethernet Cable
- Laptop (for OS installation)

📄 Software Required

- **Raspberry Pi Imager**
- Raspbian OS (Raspberry Pi OS)

🔍 Important Raspberry Pi Components

Component	Function
GPIO Pins	Used to connect sensors and actuators
HDMI Port	Connect to monitor
USB Ports	Connect keyboard/mouse
Ethernet Port	Internet connection
MicroSD Slot	Stores OS
Power Port	Supplies power

🧭 Procedure (Step-by-Step)

Step 1: Install Raspberry Pi OS

1. Insert MicroSD card into laptop.
2. Open **Raspberry Pi Imager**.
3. Select:
    - OS → Raspberry Pi OS (32-bit)
    - Storage → Select SD card
4. Click **Write**.
5. Wait until flashing completes.

Step 2: Insert SD Card

1. Remove SD card safely.
2. Insert it into Raspberry Pi SD slot.

Step 3: Connect Hardware

1. Connect HDMI to monitor.
2. Connect keyboard and mouse via USB.
3. Connect Ethernet cable.
4. Connect power supply.

Step 4: Boot System

1. Raspberry Pi will automatically boot.
2. Follow setup wizard:
    - Select Country
    - Set Username & Password
    - Connect WiFi (if needed)
    - Update system

Step 5: Open Terminal

Type:

```
sudo apt update
sudo apt upgrade
```

This updates system packages.

📄 Basic Commands to Familiarise

```
pwd       # Show current directory
ls        # List files
cd Desktop # Change directory
mkdir test # Create folder
```

📊 Result

Raspberry Pi OS was successfully installed and system booted properly. Hardware components and interfaces were identified and tested.

▯ Conclusion

Thus, Raspberry Pi was successfully set up with Raspbian OS and basic interfacing was understood.

1.

Displaying Different LED Patterns with Raspberry Pi

▯ Aim

To interface multiple LEDs with Raspberry Pi and display different LED patterns using GPIO programming in Python.

- Raspberry Pi (Model 3/4)
- 4 LEDs
- 4 Resistors (220Ω)
- Breadboard
- Jumper Wires
- Monitor, Keyboard, Mouse

🔌 Circuit Connections

LED	GPIO Pin	Physical Pin
LED1	GPIO17	Pin 11
LED2	GPIO18	Pin 12
LED3	GPIO27	Pin 13

LED	GPIO Pin	Physical Pin
LED4	GPIO22	Pin 15

- Connect LED anode (+) to GPIO pin through 220Ω resistor
- Connect cathode (-) to GND (Pin 6)

□ Procedure

Step 1: Boot Raspberry Pi

Start Raspberry Pi with Raspbian OS.

Step 2: Enable GPIO Library

Open terminal and ensure GPIO library is installed:

pip3 install RPi.GPIO

Step 3: Create Python File

nano led_patterns.py

Step 4: Write the Integrated Code

```
import RPi.GPIO as GPIO
import time

# Use BCM GPIO numbering
GPIO.setmode(GPIO.BCM)

# Define LED pins
leds = [17, 18, 27, 22]

# Setup pins as output
for led in leds:
    GPIO.setup(led, GPIO.OUT)

try:
    while True:
        # Pattern 1: All LEDs ON
        for led in leds:
            GPIO.output(led, True)
        time.sleep(1)

        # Pattern 2: All LEDs OFF
        for led in leds:
            GPIO.output(led, False)
        time.sleep(1)

        # Pattern 3: Running Light
        for led in leds:
```

```
        GPIO.output(led, True)
        time.sleep(0.3)
        GPIO.output(led, False)

    # Pattern 4: Alternate LEDs
    GPIO.output(17, True)
    GPIO.output(27, True)
    GPIO.output(18, False)
    GPIO.output(22, False)
    time.sleep(1)

    GPIO.output(17, False)
    GPIO.output(27, False)
    GPIO.output(18, True)
    GPIO.output(22, True)
    time.sleep(1)

except KeyboardInterrupt:
  GPIO.cleanup()
```

Step 5: Save and Exit

Press:
CTRL + X → Y → Enter

Step 6: Run Program

python3 led_patterns.py

📊 **Output**

- LEDs blink together
- LEDs turn OFF together
- Running light pattern observed
- Alternate LED blinking pattern displayed

□ Result

Different LED patterns were successfully generated using Raspberry Pi GPIO programming.

📓 **Conclusion:**Thus, LEDs were successfully interfaced with Raspberry Pi and various lighting patterns were implemented using Python.

2 – Displaying Time over 4-Digit 7-Segment Display using Raspberry Pi

Displaying Time over 4-Digit 7-Segment Display using Raspberry Pi.

Apparatus:

Hardware Required:

1. **Raspberry Pi 3B+/4**
2. **Micro SD Card with Raspberry Pi OS**
3. **5V Power Adapter**
4. **Monitor**
5. **USB Keyboard and Mouse**
6. **4-Digit 7-Segment Display (TM1637)**
7. **Jumper Wires**

Software Required:

1. **Raspberry Pi OS**
2. **Thonny Python IDE**
3. **tm1637 Library**

Procedure:

1. Hardware Setup:

- **Connect VCC of TM1637 to 5V.**
- **Connect GND to GND.**
- **Connect CLK to GPIO21.**
- **Connect DIO to GPIO20.**

2. Install Required Library:

Open Terminal and type:

pip3 install tm1637

Program 1: Testing TM1637 Display

(This program checks whether the display is working properly.)

```
import tm1637
import time

display = tm1637.TM1637(clk=21, dio=20)
display.brightness(2)

while True:
    display.show([1, 2, 3, 4])
    time.sleep(1)
    display.show([4, 3, 2, 1])
    time.sleep(1)
```

Program 2: Displaying Current Time

```
import tm1637
import time
from datetime import datetime

display = tm1637.TM1637(clk=21, dio=20)
display.brightness(2)

while True:
    now = datetime.now()
    hour = now.hour
    minute = now.minute

    display.show([hour // 10, hour % 10,
                  minute // 10, minute % 10])
    time.sleep(1)
```

Output:

- **In Program 1, numbers 1234 and 4321 are displayed alternately.**
- **In Program 2, the current system time is displayed in HHMM format.**
- **Time updates every second automatically.**

Conclusion:

In this practical, the TM1637 4-digit 7-segment display was successfully interfaced with Raspberry Pi. The display was first tested using a sample program and then used to show real-time system clock using Python programming.

03

Aim:

Raspberry Pi Based Oscilloscope using ADS1115.

Apparatus:

Hardware Required:

1. Raspberry Pi 3B+/4
2. Micro SD Card with Raspberry Pi OS
3. 5V 3A Power Adapter
4. Monitor
5. USB Keyboard and Mouse
6. ADS1115 ADC Module
7. Potentiometer (for analog input testing)
8. Female–Female Jumper Wires
9. Breadboard

Software Required:

1. Raspberry Pi OS
2. Thonny Python IDE
3. adafruit-circuitpython-ads1x15 Library
4. matplotlib Library

Procedure:

1. Hardware Setup:

Connect ADS1115 to Raspberry Pi using I2C:

ADS1115 Pin	Raspberry Pi Pin
VDD	3.3V
GND	GND
SDA	GPIO2 (Pin 3)
SCL	GPIO3 (Pin 5)
ADDR	GND

ADS1115 Pin	Raspberry Pi Pin
A0	Potentiometer middle pin

Potentiometer:

- One side → 3.3V
- Other side → GND

---

2. Enable I2C:

Open terminal:

sudo raspi-config

→ Interface Options → I2C → Enable → Reboot

---

3. Install Required Libraries:

```
pip3 install adafruit-blinka
pip3 install adafruit-circuitpython-ads1x15
pip3 install matplotlib
```

Program:

```
import time

import board

import busio

import adafruit_ads1x15.ads1115 as ADS # This is the module

from adafruit_ads1x15.analog_in import AnalogIn


# Initialize I2C bus

i2c = busio.I2C(board.SCL, board.SDA)


# Create the ADC object

# Note: we use the ADS1115 class inside the ADS module

ads = ADS.ADS1115(i2c)


# FIX: Use ADS.P0 directly from the module level
```

```
chan = AnalogIn(ads, 0)

print("--- Oscilloscope Practical Started ---")

try:
    while True:
        volts = chan.voltage
        # Create a text-based waveform
        # Scaling 0-3.3V to 0-50 characters
        pattern = "?" * int((volts / 3.3) * 50)

        print(f"{volts:5.3f} V | {pattern}")
        time.sleep(0.1)

except KeyboardInterrupt:
    print("\nPractical Finished.")
```

Output:

- A live waveform graph is displayed on the screen.

Conclusion:

In this practical, ADS1115 ADC module was successfully interfaced with Raspberry Pi using I2C communication. Analog signals were read and displayed graphically using Python and Matplotlib, converting Raspberry Pi into a basic digital oscilloscope.

04

Aim:

Controlling Raspberry Pi with Telegram Bot.

Apparatus:

Hardware Required:

1. Raspberry Pi 3B+/4
2. Micro SD Card with Raspberry Pi OS
3. 5V 3A Power Adapter

4. Monitor
5. USB Keyboard and Mouse
6. Relay Module / LED
7. Breadboard
8. Female–Male Jumper Wires
9. Internet Connection

Software Required:

1. Raspberry Pi OS
2. Telegram App (Android/iOS)
3. Python3
4. telepot Library

Procedure:

1. Hardware Setup:

- Connect Relay IN pin to GPIO20.
- Connect VCC to 5V.
- Connect GND to Ground.
- If using LED instead of relay:
    - LED anode → GPIO20
    - LED cathode → GND through 220Ω resistor

---

2. Create Telegram Bot:

1. Open Telegram app.
2. Search for **BotFather**.
3. Send command:

/start

4. Send:

/newbot

5. Enter bot name.
6. Enter unique username.
7. Copy the **HTTP API Token** provided.

---

3. Install Required Library:

Open terminal and type:

```
sudo apt update
sudo apt upgrade
pip3 install telepot
```

Program:

Replace "YOUR_BOT_TOKEN" with your actual token.

```
import RPi.GPIO as GPIO
import telepot
from time import sleep

TOKEN = "YOUR_BOT_TOKEN"

GPIO.setmode(GPIO.BCM)
GPIO.setup(20, GPIO.OUT)

def handle(msg):
    chat_id = msg['chat']['id']
    command = msg['text']

    if command == '/on':
        GPIO.output(20, True)
        bot.sendMessage(chat_id, "Device Turned ON")

    elif command == '/off':
        GPIO.output(20, False)
        bot.sendMessage(chat_id, "Device Turned OFF")

bot = telepot.Bot(TOKEN)
bot.message_loop(handle)

print("Bot is Running...")

while True:
    sleep(10)
```

OR THIS:

```
import RPi.GPIO as GPIO

import telepot
```

```
from telepot.loop import MessageLoop
import time

# Pin Setup
LED_PIN = 17
GPIO.setmode(GPIO.BCM)
GPIO.setup(LED_PIN, GPIO.OUT)

def handle(msg):
    chat_id = msg['chat']['id']
    command = msg['text'].lower()

    print(f'Received: {command}')

    if command == '/on':
        GPIO.output(LED_PIN, GPIO.HIGH)
        bot.sendMessage(chat_id, "LED is now ON")
    elif command == '/off':
        GPIO.output(LED_PIN, GPIO.LOW)
        bot.sendMessage(chat_id, "LED is now OFF")
    else:
        bot.sendMessage(chat_id, "Use /on or /off to control the LED")

# Replace with your actual Token
bot = telepot.Bot('8395999857:AAE8VoyBXWz4KAjIuCQ4L-f7OroB9n1r0po')

MessageLoop(bot, handle).run_as_thread()
print('Bot is listening...')
try:
    while 1:
        time.sleep(10)
```

finally:

```
GPIO.cleanup()
```

Conclusion:

In this practical, Raspberry Pi was successfully controlled remotely using Telegram Bot. Internet-based communication was established between mobile application and Raspberry Pi, demonstrating IoT-based remote device control.

05

Aim

Raspberry Pi GPS Module Interfacing.

Apparatus

Hardware Required

1. Raspberry Pi 3B+/4
2. Micro SD Card with Raspberry Pi OS
3. 5V 3A Power Adapter
4. Monitor
5. USB Keyboard
6. USB Mouse
7. GPS Module (Neo-6M or similar)
8. Female–Female Jumper Wires
9. Breadboard

Software Required

1. Raspberry Pi OS
2. Thonny Python IDE
3. Python3
4. Serial Library

Procedure

1. Hardware Setup

Connect the GPS module to Raspberry Pi UART pins.

GPS Module Pin	Raspberry Pi
VCC	5V (Pin 2)
GND	GND (Pin 6)
TX	GPIO15 (RX – Pin 10)
RX	GPIO14 (TX – Pin 8)

Ensure all connections are properly made before powering the Raspberry Pi.

---

2. Enable Serial Communication

1. Open terminal.
2. Run the command:

sudo raspi-config

3. Navigate to:

Interface Options → Serial Port

4. Disable **login shell over serial**.
5. Enable **serial port hardware**.
6. Reboot the Raspberry Pi.

---

3. Install Required Libraries

Open terminal and run:

pip3 install pyserial

```
def GPS_Info():
    global NMEA_buff
    global lat_in_degrees
    global long_in_degrees

    # Check if we have enough fields and if the lat/long fields are not empty
    if len(NMEA_buff) > 5 and NMEA_buff[1] != '' and NMEA_buff[3] != '':
        nmea_time = NMEA_buff[0]
```

```
    nmea_latitude = NMEA_buff[1]
    nmea_longitude = NMEA_buff[3]

    print("NMEA Time: ", nmea_time,'\n')
    print ("NMEA Latitude:", nmea_latitude,"NMEA Longitude:", nmea_longitude,'\n')

    try:
        lat = float(nmea_latitude)
        longi = float(nmea_longitude)

        lat_in_degrees = convert_to_degrees(lat)
        long_in_degrees = convert_to_degrees(longi)
    except ValueError:
        print("Wait... data was corrupted or incomplete.")
else:
    print("Waiting for Satellite Fix... (Ensure you are near a window or outdoors)")
    lat_in_degrees = "0"
    long_in_degrees = "0"
```

Conclusion

In this practical, the GPS module was successfully interfaced with Raspberry Pi using serial communication. The system was able to receive satellite data, extract latitude and longitude coordinates, and display the location on Google Maps, demonstrating a basic IoT-based location tracking system.

06

Aim

Interfacing Raspberry Pi with RFID.

Apparatus

Hardware Required

1. Raspberry Pi 3B+/4
2. Micro SD Card with Raspberry Pi OS
3. 5V Power Adapter

4. Monitor
5. USB Keyboard
6. USB Mouse
7. RFID Reader Module (RC522)
8. RFID Tag/Card
9. Jumper Wires
10. Breadboard
11. LED (optional)
12. Buzzer (optional)

Software Required

1. Raspberry Pi OS
2. Thonny Python IDE
3. Python3
4. SimpleMFRC522 Library

Procedure

1. Hardware Setup

Connect the **RFID RC522 module** to Raspberry Pi using SPI interface.

RC522 Pin	Raspberry Pi
SDA	GPIO8 (Pin 24)
SCK	GPIO11 (Pin 23)
MOSI	GPIO10 (Pin 19)
MISO	GPIO9 (Pin 21)
IRQ	Not Connected
GND	GND (Pin 6)
RST	GPIO25 (Pin 22)
3.3V	3.3V (Pin 1)

⚠ Important: RC522 works only with **3.3V**, not 5V.

---

2. Enable SPI Interface

Open terminal and run:

sudo raspi-config

Navigate to:

Interface Options → SPI → Enable

Restart Raspberry Pi.

---

3. Install Required Libraries

Open terminal and run:

```
sudo apt update
sudo apt upgrade
pip3 install mfrc522
```

Program 1: Writing Data to RFID Tag

```
import RPi.GPIO as GPIO
from mfrc522 import SimpleMFRC522

reader = SimpleMFRC522()

try:
    text = input("Enter data to write: ")
    print("Place the RFID tag near the reader")
    reader.write(text)
    print("Data written successfully")

finally:
    GPIO.cleanup()
```

Program 2: Reading Data from RFID Tag

```
import RPi.GPIO as GPIO
from mfrc522 import SimpleMFRC522

reader = SimpleMFRC522()

try:
    print("Place the RFID tag near the reader")
    id, text = reader.read()
    print("Tag ID:", id)
    print("Stored Data:", text)
```

```
finally:
    GPIO.cleanup()
```

Conclusion

In this practical, the RFID RC522 reader was successfully interfaced with Raspberry Pi using SPI communication. Data was written to the RFID tag and retrieved successfully, demonstrating RFID-based identification and authentication systems.

07

Aim

Fingerprint Sensor Interfacing with Raspberry Pi.

Apparatus

Hardware Required

1. Raspberry Pi 3B+/4
2. Micro SD Card with Raspberry Pi OS
3. 5V Power Adapter
4. Monitor
5. USB Keyboard
6. USB Mouse
7. Fingerprint Sensor Module (R305 / R307 / GT511)
8. Jumper Wires
9. Breadboard

Software Required

1. Raspberry Pi OS
2. Thonny Python IDE
3. Python3
4. PyFingerprint Library

Procedure

1. Hardware Setup

Connect the fingerprint sensor to Raspberry Pi using UART communication.

Fingerprint Sensor Pin	Raspberry Pi
VCC	5V (Pin 2)
GND	GND (Pin 6)
TX	GPIO15 (RX – Pin 10)
RX	GPIO14 (TX – Pin 8)

Ensure all connections are secure before powering the Raspberry Pi.

---

2. Enable Serial Communication

Open terminal and run:

sudo raspi-config

Navigate to:

Interface Options → Serial Port

- Disable **login shell over serial**
- Enable **serial port hardware**

Restart the Raspberry Pi.

---

3. Install Required Library

Install the fingerprint library using:

pip3 install pyfingerprint

```
from pyfingerprint.pyfingerprint import PyFingerprint


try:
    f = PyFingerprint('/dev/ttyS0', 57600, 0xFFFFFFFF, 0x00000000)


    if (f.verifyPassword() == False):
        raise ValueError('The given fingerprint sensor password is wrong!')


except Exception as e:
    print('The fingerprint sensor could not be initialized!')
    print('Exception message: ' + str(e))
```

```
    exit(1)

print('Waiting for finger...')

try:
    while (f.readImage() == False):
        pass

    f.convertImage(0x01)

    result = f.searchTemplate()

    positionNumber = result[0]

    if (positionNumber == -1):
        print('No match found!')
    else:
        print('Fingerprint matched at position #' + str(positionNumber))

except Exception as e:
    print('Operation failed!')
    print('Exception message: ' + str(e))
```

Conclusion

In this practical, the fingerprint sensor was successfully interfaced with the Raspberry Pi using serial communication. The system captured fingerprint images and compared them with stored templates, demonstrating a biometric authentication system using Raspberry Pi.

08

Aim

Installing Windows 10 IoT Core on Raspberry Pi.

Apparatus

Hardware Required

1. Raspberry Pi 3B+/4
2. Micro SD Card (8GB or higher)
3. SD Card Reader
4. Monitor
5. HDMI Cable
6. USB Keyboard
7. USB Mouse
8. Ethernet Cable / WiFi
9. Power Adapter

Software Required

1. Windows 10 Operating System
2. Windows 10 IoT Core Dashboard
3. Raspberry Pi Firmware
4. Micro SD Card Formatter

Procedure

1. Download Windows 10 IoT Core Dashboard

1. Open a web browser on the computer.
2. Go to the official Microsoft website.
3. Download **Windows 10 IoT Core Dashboard**.
4. Install the application on the computer.

---

2. Prepare the Micro SD Card

1. Insert the Micro SD card into the computer using an SD card reader.
2. Open **Windows 10 IoT Core Dashboard**.
3. Select **Set up a new device**.

---

3. Configure Device Settings

Enter the following details:

- **Device Type:** Raspberry Pi

- **OS Build:** Windows 10 IoT Core
- **Drive:** Select the inserted SD card
- **Device Name:** RaspberryPi
- **Administrator Password:** Set a password
- **WiFi Network:** Enter network credentials (optional)

---

4. Install the Operating System

1. Click **Download and Install**.
2. The dashboard downloads the OS and flashes it to the SD card.
3. Wait until the installation process is completed.

---

5. Boot Raspberry Pi

1. Remove the Micro SD card from the computer.
2. Insert it into the Raspberry Pi.
3. Connect the monitor, keyboard, and mouse.
4. Connect the power supply.

The Raspberry Pi will boot into **Windows 10 IoT Core**.

Output

- Windows 10 IoT Core successfully boots on the Raspberry Pi.
- The device information and IP address are displayed on the screen.
- The Raspberry Pi can be managed using the **Windows IoT Core Dashboard**.

Example display:

Windows 10 IoT Core
Device Name: RaspberryPi
IP Address: 192.168.1.20

Conclusion

In this practical, Windows 10 IoT Core was successfully installed on the Raspberry Pi using the Windows IoT Core Dashboard. The Raspberry Pi booted into the Windows IoT Core environment and was successfully configured for IoT application development.